⏱ Promoción de Año Nuevo – Últimas horas - 10 hours only!

20th Sep 2023 | Lectura de 22 minutos

20 ejemplos de consultas SQL básicas para principiantes: Una visión completa



Tihomir Babic

SQL Consultas De SQL

Índice

- ¿Qué es SQL
- Conjunto de datos
  - 1. Selecc todas l colum de una tabla
    - C
    - E
    - S
  - 2. Selecc de una

Estas 20 consultas básicas son imprescindibles en un paquete de inicio para todo principiante de SQL. Estos ejemplos le ayudarán en su viaje hacia el dominio de SQL.

Te has propuesto aprender SQL, has buscado en Google "ejemplos de consultas SQL básicas" o algo similar, y aquí estás, mirando este artículo. ¿Y ahora qué? Todo aprendizaje empieza por lo básico, así que empecemos por lo más básico:

¿Qué es SQL?

Lo primero es saber qué es SQL. SQL, o Lenguaje de Consulta Estructurado, es un lenguaje de programación. Como cualquier lenguaje -de programación o natural- se utiliza para comunicarse, para hablar. SQL está diseñado para hablar con una

colum
de una
tabla
- C
- E
- S

3. Selecc
de dos
colum
de una
tabla
- C
- E
- S

4. Selecc
de dos
más)
colum
de una
tabla y
filtrado
media
compa
numér

base de datos. Lo hacemos mediante sentencias que llamamos consultas, que son comandos SQL para recuperar datos de la base de datos.

Pronto te mostraremos 20 ejemplos de consultas SQL básicas para empezar a hablar con la base de datos. Todas estas consultas se enseñan en nuestro SQL para principiantes curso; este curso te dará aún más estructura, ejemplos y desafíos para resolver. Tiene 129 ejercicios interactivos sobre consultas a una o más tablas, agregación y agrupación de datos, JOINs, subconsultas y operaciones de conjunto. Incluso con los 20 próximos ejemplos, no mostraremos todos los detalles ni siquiera todas las consultas de nivel básico. Por eso recomendamos usar el curso como una plataforma para practicar los fundamentos que discutiremos aquí.

Además, la mayoría de nuestros ejemplos están bien presentados en nuestra SQL para principiantes Cheat Sheet. Siéntase libre de tenerla a su lado - podría ayudarle a entender mejor lo que sigue a continuación.

> Si te gusta aprender SQL usando ejercicios prácticos, entonces tienes que probar LearnSQL.es.

No perdamos tiempo. Presentaremos el conjunto de datos y, a continuación, empezaremos a escribir y explicar las consultas SQL básicas.

Conjunto de datos

El conjunto de datos consta de dos tablas. La primera se muestra a continuación; puedes crear esta tabla copiando y ejecutando esta consulta desde GitHub.

id	first_name	last_name	department	salar
1	Paul	Garrix	Corporate	3,547
2	Astrid	Fox	Private Individuals	2,845
3	Matthias	Johnson	Private Individuals	3,009
4	Lucy	Patterson	Private Individuals	3,547
5	Tom	Page	Corporate	5,974
6	Claudia	Conte	Corporate	4,714
7	Walter	Deer	Private Individuals	3,547
8	Stephanie	Marx	Corporate	2,894
9	Luca	Pavarotti	Private Individuals	4,123
10	Victoria	Pollock	Corporate	4,789

Como cualquier tabla, tiene un nombre: `employees`. Cada tabla tiene columnas que también tienen nombres. Describen qué datos contiene cada columna.

Las columnas y los datos de la tabla anterior son:

- `id` - El ID único del empleado y la clave primaria de la tabla.
- `first_name` - El nombre del empleado.
- `last_name` - El apellido del empleado.
- `department` - El departamento del empleado.
- `salary` - El salario mensual del empleado, en USD.

Todo esto nos indica que esta tabla es una lista de los empleados de una empresa y sus salarios. También hay datos sobre los departamentos de los empleados. Todos los empleados trabajan en la división de ventas, donde el departamento puede ser Corporativo o Privado Individuals. En otras palabras, los empleados venden los productos de la empresa a empresas y particulares.

La otra tabla del conjunto de datos se llama `quarterly_sales`. Se muestra a continuación, y la consulta para crearla está aquí.

employee_id	q1_2022	q2_2022	q3_2022	q
8	3,471.41	14,789.25	3,478.34	1
4	5,417.81	12,846.23	8,741.54	3
10	1,547.52	1,269.66	1,478.65	2
1	8,715.55	8,465.65	24,747.82	3
3	12,774.51	24,784.31	12,223.34	8
2	4,989.23	5,103.22	4,897.98	5
7	18,415.66	15,279.37	14,634.44	1
6	2,498.63	8,741.45	3,997.65	2
5	6,349.74	7,555.55	6,944.35	7
9	4,485.36	4,101.50	8,787.45	7

Las columnas son:

- `employee_id` - El ID único del empleado. Además, una clave externa que hace referencia a la columna id de la tabla `employees`.
- `q1_2022` - Las ventas realizadas por ese empleado en el primer trimestre de 2022.
- `q2_2022` - Las ventas realizadas por ese empleado en el segundo trimestre de 2022.
- `q3_2022` - Las ventas realizadas por ese empleado en el tercer trimestre de 2022.
- `q4_2022` - Las ventas realizadas por ese empleado en el cuarto trimestre de 2022.

En general, esta tabla es una lista de las ventas de cada trimestre realizadas por cada empleado mostrado en la primera tabla.

Ahora, empecemos a escribir las consultas SQL.

1. Seleccionar todas las columnas de una tabla

Esta consulta es útil cuando quieres obtener rápidamente todas las columnas de una tabla sin escribir cada columna en la sentencia SELECT.

Consulta

Code

```
SELECT *
FROM employees;
```

Explicación

Siempre que quiera seleccionar cualquier número de columnas de cualquier tabla, necesita utilizar la sentencia `SELECT` . Se escribe, de forma bastante obvia, utilizando la palabra clave `SELECT` .

Después de la palabra clave viene un asterisco (`*`), que es la abreviatura de "todas las columnas de la tabla".

Para especificar la tabla, utilice la cláusula `FROM` y escriba después el nombre de la tabla.

Salida

La salida de la consulta es la tabla completa `employees` como se muestra a continuación.

id	first_name	last_name	department	salar
1	Paul	Garrix	Corporate	3,547
2	Astrid	Fox	Private Individuals	2,845
3	Matthias	Johnson	Private Individuals	3,009
4	Lucy	Patterson	Private Individuals	3,547
5	Tom	Page	Corporate	5,974
6	Claudia	Conte	Corporate	4,714
7	Walter	Deer	Private Individuals	3,547

id	first_name	last_name	department	salar
8	Stephanie	Marx	Corporate	2,894
9	Luca	Pavarotti	Private Individuals	4,123
10	Victoria	Pollock	Corporate	4,789

2. Selección de una columna de una tabla

Puede utilizar esta consulta cuando sólo necesite una columna de la tabla...

Consulta

Code

```
SELECT first_name
FROM employees;
```

Explicación

El planteamiento es similar al de la consulta anterior. Sin embargo, esta vez, en lugar de un asterisco, escribimos el nombre específico de la columna en `SELECT`. En este caso, es la columna `first_name`.

La segunda línea de la consulta es la misma: hace referencia a la tabla en la cláusula `FROM`.

Salida

La consulta devuelve la lista de los nombres de pila de los empleados.

first_name
Paul
Astrid
Matthias
Lucy
Tom
Claudia
Walter
Stephanie
Luca
Victoria

3. Selección de dos columnas de una tabla

Esta consulta es útil para seleccionar dos (o más) columnas de una tabla.

Consulta

Code

```
SELECT first_name,
     last_name
FROM employees;
```

Explicación

De nuevo, el planteamiento es similar al de los ejemplos anteriores. Para seleccionar dos columnas, debe escribir sus nombres en `SELECT`. Lo importante es que las columnas estén separadas por una coma. Puede ver en el ejemplo que hay una coma entre las columnas `first_name` y `last_name`.

A continuación, como de costumbre, haz referencia a la tabla `employees` en `FROM`.

Salida

Ahora la consulta muestra los nombres completos de los empleados.

first_name	last_name
Paul	Garrix
Astrid	Fox
Matthias	Johnson
Lucy	Patterson
Tom	Page
Claudia	Conte
Walter	Deer
Stephanie	Marx
Luca	Pavarotti
Victoria	Pollock

4. Selección de dos (o más) columnas de una tabla y filtrado mediante comparación numérica en WHERE

Conocer esta consulta SQL le permitirá filtrar datos según valores numéricos. Puede hacerlo utilizando operadores de comparación en la cláusula `WHERE` .

Aquí está la visión general de los operadores de comparación SQL.

Comparison Operator	Description
=	Is equal to
>	Is greater than
<	Is less than
>=	Is greater than or equal to
<=	Is less than or equal to
<>	Is not equal to

Consulta

Code

```
SELECT
  first_name,
  last_name,
  salary
FROM employees
WHERE salary > 3800;
```

Explicación

En realidad, la consulta selecciona tres columnas, no dos. Es lo mismo que con dos columnas: basta con escribirlas en `SELECT` y separarlas con comas.

Luego hacemos referencia a la tabla en `FROM` .

Ahora, necesitamos mostrar sólo los empleados con un salario superior a 3.800. Para ello, es necesario utilizar `WHERE` . Es una cláusula que acepta condiciones y se utiliza para filtrar la salida. Recorre la tabla y devuelve sólo los datos que satisfacen la condición.

En nuestro caso, buscamos salarios "mayores que" un determinado número. En otras palabras, una condición que utiliza el operador de comparación >.

Para establecer la condición, escribimos el nombre de la columna en `WHERE` . Después viene el operador de comparación y, a continuación, el valor que debe tener el dato mayor que. Esta condición nos devolverá ahora todos los salarios que sean superiores a 3.800.

Salida

La consulta devuelve cuatro empleados y sus salarios. Como puedes ver, todos tienen sueldos superiores a 3.800.

first_name	last_name	salary
Tom	Page	5,974.41
Claudia	Conte	4,714.12
Luca	Pavarotti	4,123.45
Victoria	Pollock	4,789.53

5. Selección de Dos Columnas y Filtrado Utilizando una Condición de Igualdad en WHERE

Una vez más, este ejemplo de consulta SQL básica es útil cuando se desea seleccionar varias columnas pero no todas las filas de la tabla. Ahora quiere encontrar los valores que son iguales al valor de la condición. Para ello, necesita la condición de igualdad (`=`).

Consulta

Code

```
SELECT
  first_name,
  last_name
FROM employees
WHERE first_name = 'Luca';
```

Explicación

La consulta selecciona los nombres y apellidos de los empleados.

Sin embargo, queremos mostrar sólo los empleados que se llaman Luca. Para ello, volvemos a utilizar `WHERE` . El planteamiento es similar al del ejemplo anterior: utilizamos WHERE, escribimos el nombre de la columna y utilizamos el operador de comparación. Esta vez, nuestra condición utiliza el signo igual (`=`).

En otras palabras, los valores de la columna `first_name` tienen que ser iguales a Luca. Además, cuando la condición no es un número, sino un texto o una fecha/hora, debe escribirse entre comillas simples (`''`). Por eso nuestra condición se escribe como ' `Luca` ', no simplemente `Luca` .

Salida

La salida muestra que sólo hay un empleado llamado Luca, y su nombre completo es Luca Pavarotti.

first_name	last_name
Luca	Pavarotti

6. Selección de dos columnas y ordenación por una columna

He aquí otro ejemplo de consulta SQL básica que le resultará útil. Se puede utilizar siempre que tenga que ordenar la salida de una determinada manera para hacerla más legible.

Ordenar o clasificar la salida se hace utilizando la cláusula `ORDER BY` . Por defecto, ordena la salida en orden ascendente. Esto funciona alfabéticamente (para datos de texto), del número más bajo al más alto (para datos numéricos), o de la fecha u hora más antigua a la más reciente (para fechas y horas).

Consulta

Code

```
SELECT
  first_name,
  last_name
FROM employees
ORDER BY last_name;
```

Explicación

Volvemos a seleccionar los nombres y apellidos de los empleados. Pero ahora queremos ordenar la salida de una manera específica. En este ejemplo, por los apellidos de los empleados. Para ello, utilizamos `ORDER BY` . En él, simplemente escribimos el nombre de la columna.

Una excelente manera de empezar a aprender SQL es con un curso de SQL en línea. Busca uno que te permita usar el lenguaje de inmediato. LearnSQL.es ofrece un fantástico curso de SQL para principiantes que hace que aprender SQL sea rápido y fácil. Sus 129 ejercicios interactivos te brindan experiencia práctica trabajando con bases de datos y declaraciones SQL.

Podríamos añadir la palabra clave `ASC` después de eso para ordenar la salida de forma ascendente. Sin embargo, esto no es obligatorio, ya que la ordenación ascendente es la predeterminada en SQL.

Salida

La consulta devuelve una lista de empleados ordenados alfabéticamente por sus apellidos.

first_name	last_name
Claudia	Conte
Walter	Deer
Astrid	Fox

first_name	last_name
Paul	Garrix
Matthias	Johnson
Stephanie	Marx
Tom	Page
Lucy	Patterson
Luca	Pavarotti
Victoria	Pollock

7. Selección de Dos Columnas y Ordenación Descendente por una Columna

Este ejemplo es similar al anterior y tiene el mismo propósito: ordenar la salida de su consulta SQL. Sin embargo, en este caso, los datos se ordenan de forma descendente (de Z a A, de 10 a 1).

Consulta

Code

```
SELECT
  first_name,
  last_name
FROM employees
ORDER BY last_name DESC;
```

Explicación

La consulta es casi exactamente la misma que en el ejemplo anterior. La única diferencia es que estamos ordenando la salida por el nombre del empleado de forma descendente.

Para ello, ponga la palabra clave `DESC` después de la columna `last_name` en la cláusula `ORDER BY` .

Salida

first_name	last_name
Victoria	Pollock
Luca	Pavarotti
Lucy	Patterson
Tom	Page

first_name	last_name
Stephanie	Marx
Matthias	Johnson
Paul	Garrix
Astrid	Fox
Walter	Deer
Claudia	Conte

Puede ver que la salida está ordenada como queríamos.

8. Seleccionando Dos Columnas de una Tabla y Ordenando Descendentemente por Dos Columnas

Ordenar una consulta SQL puede ser más sofisticado. Es habitual ordenar datos por dos o más columnas, algo con lo que probablemente ya estés familiarizado como usuario de Excel o Google Sheets. Lo mismo se puede hacer en SQL.

Consulta

Code

```
SELECT
  first_name,
  last_name,
  salary
FROM employees
ORDER BY salary DESC, last_name ASC;
```

Explicación

Con esta consulta, estamos construyendo sobre el ejemplo anterior; queremos ordenar la salida por el salario del empleado *y* su apellido. Esta vez, ordenamos por salario de forma descendente y luego por apellido de forma ascendente.

Hacemos referencia a la columna salario en `ORDER BY` y la seguimos con la palabra clave `DESC`. La palabra clave `DESC` indica orden descendente. Antes del segundo criterio de ordenación, tenemos que poner una coma. Después viene el segundo criterio/columna, que en este caso es `last_name`. Puede añadir u omitir la palabra clave ASC para ordenar la salida en orden ascendente.

Nota: ¡El **orden de las columnas en ORDER BY es importante!** La consulta escrita como está arriba ordenará primero por salario de forma descendente y luego por apellido de forma ascendente. Si escribiera `ORDER BY last_name ASC, salary DESC`, ordenaría primero por el apellido y luego por el salario en orden descendente.

Salida

first_name	last_name	salary
Tom	Page	5,974.41
Victoria	Pollock	4,789.53
Claudia	Conte	4,714.12
Luca	Pavarotti	4,123.45
Walter	Deer	3,547.25
Paul	Garrix	3,547.25
Lucy	Patterson	3,547.25
Matthias	Johnson	3,009.41
Stephanie	Marx	2,894.51
Astrid	Fox	2,845.56

La salida está ordenada por salario. Cuando el salario es el mismo (filas verdes), los datos se ordenan alfabéticamente por apellido.

9. Selección de dos columnas con una condición lógica compleja en WHERE

Este ejemplo demostrará nuevamente como filtrar la salida utilizando WHERE. Esta vez será un poco más avanzado, ya que utilizaremos un operador lógico. En SQL, los operadores lógicos le permiten probar si la condición de filtrado es verdadera o no. También permiten establecer múltiples condiciones.

Los tres operadores lógicos básicos en SQL son AND, OR y NOT. En la siguiente consulta, utilizaremos OR para obtener salarios inferiores a 3.000 o superiores a 5.000.

Consulta

Code

```
SELECT
  first_name,
  last_name,
  salary
FROM employees
WHERE salary > 5000 OR salary < 3000;
```

Explicación

Usamos esta consulta para seleccionar el nombre, apellido y salario del empleado de la tabla `employees` .

Sin embargo, queremos mostrar sólo aquellos empleados cuyos salarios sean superiores a 5.000 dólares o inferiores a 3.000 dólares. Para ello utilizamos el operador lógico OR y los operadores de comparación de `WHERE` .

Escribimos la primera condición en `WHERE` , donde hacemos referencia a la columna `salary` y establecemos la condición de que los valores deben ser superiores a 5.000 dólares. A continuación, utilizamos el operador `OR` , seguido de la segunda condición. La segunda condición vuelve a hacer referencia a la columna del salario y utiliza el operador "menor que" para devolver los valores inferiores a 3.000.

Salida

first_name	last_name	salary
Astrid	Fox	2,845.56
Tom	Page	5,974.41
Stephanie	Marx	2,894.51

La consulta devuelve sólo tres empleados y sus salarios, ya que son los únicos que cumplen las condiciones.

10. Cálculos sencillos sobre columnas

En este ejemplo, mostraremos como se pueden realizar operaciones matemáticas sencillas sobre las columnas de la tabla.

Utilizaremos uno de los operadores aritméticos de SQL.

Arithmetic Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo, i.e. returns the remainder of the integer division.

Consulta

Code

```
SELECT
  employee_id,
  q1_2022 + q2_2022 AS h1_2022
FROM quarterly_sales;
```

Explicación

En la consulta anterior, queremos encontrar las ventas en la primera mitad de 2022 para cada empleado.

Lo hacemos seleccionando primero la columna `employee_id` de la tabla `quarterly_sales`.

A continuación, seleccionamos la columna `q1_2022` y utilizamos el operador aritmético de suma para añadir la columna `q2_2022`. También le damos a esta nueva columna calculada un alias de `h1_2022` utilizando la palabra clave `AS`.

Salida

employee_id	h1_2022
8	18,260.66
4	18,264.04
10	2,817.18
1	17,181.20
3	37,558.82
2	10,092.45
7	33,695.03
6	11,240.08
5	13,905.29
9	8,586.86

La salida muestra todos los ID de los empleados y sus respectivas ventas en el primer semestre de 2022.

11. Utilización de SUM() y GROUP BY

Esta consulta utiliza la función agregada `SUM()` con GROUP BY. En SQL, las funciones de agregado trabajan sobre grupos de datos; por ejemplo, `SUM(sales)` muestra el total de todos los valores de la columna `sales`. Es útil conocer esta función cuando se quieren poner datos en grupos y mostrar el total de cada grupo.

Consulta

Code

```
SELECT
  department,
  SUM(salary) AS total_salaries
FROM employees
GROUP BY department;
```

Explicación

El objetivo de la consulta anterior es encontrar el importe total de los salarios de cada departamento. Esto se consigue de la siguiente manera.

Primero, seleccione la columna departamento de la tabla `employees`. A continuación, utilice la función `SUM()`. Como queremos sumar los valores salariales, especificamos la columna salario en la función. Además, le damos a esta columna calculada el alias `total_salaries`.

Por último, la salida se agrupa por la columna departamento.

Nota: Cualquier columna no agrupada que aparezca en SELECT debe aparecer también en GROUP BY. Pero esto es lógico - todo el propósito es agrupar los datos por departamento, así que por supuesto la pondremos en `GROUP BY`.

Salida

department	total_salaries
Corporate	21,919.82
Private Individuals	17,072.92

La salida muestra todos los departamentos y la suma de los costes salariales mensuales totales por departamento.

12. Usando COUNT() y GROUP BY

Aquí tenemos otra consulta SQL básica que utiliza una función agregada. Esta vez, es `COUNT()`. Puede utilizarla si desea agrupar datos y mostrar el número de ocurrencias en cada grupo.

Consulta

Code

```
SELECT
  department,
  COUNT(*) AS employees_by_department
FROM employees
GROUP BY department;
```

Explicación

Queremos mostrar el número de empleados por departamento.

Seleccione el departamento de la tabla `employees`. A continuación, utilice la función de agregación `COUNT()`. En este caso, utilizamos la versión `COUNT(*)`, que cuenta todas las filas. Le damos a la columna el alias `employees_by_department`.

Como paso final, agrupamos la salida por departamento.

Nota: `COUNT(*)` **cuenta todas las filas, incluidas las que tienen los valores** `NULL` **valores.** Si no desea incluir los posibles valores NULL en la salida, utilice la versión `COUNT(column_name)` de la función. Aquí podemos utilizar `COUNT(*)` porque sabemos que no hay valores `NULL` en la tabla.

Salida

department	employees_by_department
Corporate	5
Private Individuals	5

Hay dos departamentos, cada uno con cinco empleados.

13. Uso de AVG() y GROUP BY

La función `AVG()` calcula el valor medio. Puedes utilizar esta consulta siempre que quieras agrupar datos y mostrar el valor medio de cada grupo.

Consulta

Code

```
SELECT
  department,
  AVG(salary) AS average_salary
FROM employees
GROUP BY department;
```

Explicación

La consulta es la misma que la anterior, sólo que esta vez utilizamos la función `AVG()` , ya que queremos calcular el salario medio por departamento.

Seleccionamos el departamento, utilizamos `AVG()` con la columna `salary` , y agrupamos la salida por departamento.

Salida

department	average_salary
Corporate	4,383.96
Private Individuals	3,414.58

La salida muestra dos departamentos y sus salarios medios.

14. Usando MIN() y GROUP BY

Esta es otra consulta que combina una función agregada con `GROUP BY`. Utilícela siempre que desee encontrar los valores mínimos de cada grupo.

Consulta

Code

```
SELECT
  department,
  MIN(salary) AS minimum_salary
FROM employees
GROUP BY department;
```

Explicación

De nuevo, utilizamos la misma consulta y cambiamos sólo la función agregada.

La consulta calcula el salario mínimo por departamento.

Salida

department	minimum_salary
Corporate	2,894.51
Private Individuals	2,845.56

La salida muestra los departamentos y el salario más bajo de cada departamento.

15. Uso de MAX() y GROUP BY

Este ejemplo muestra como utilizar la función de agregado `MAX()` para mostrar el valor más alto dentro de cada grupo.

Consulta

Code

```
SELECT
  department,
  MAX(salary) AS maximum_salary
FROM employees
GROUP BY department;
```

Explicación

Utilizamos la consulta para mostrar el salario más alto de cada departamento, junto con el nombre del departamento.

Ya sabes cómo funciona. La consulta es la misma que en el ejemplo anterior, pero ahora utiliza la función `MAX()`.

Salida

department	maximum_salary
Corporate	5,974.41
Private Individuals	4,123.45

La salida nos muestra los salarios más altos del departamento de Empresas y Particulares Individuals.

> La mejor manera de aprender SQL es a través de la práctica. Prueba nuestro curso interactivo SQL para principiantes.

16. Utilizar SUM(), WHERE y GROUP BY

Esta puede parecer más complicada, pero sigue siendo una consulta SQL básica. Se utiliza cuando se desea mostrar los valores totales de cada grupo pero se desea incluir sólo filas específicas en la suma.

Consulta

Code

```
SELECT
  department,
  SUM(salary) AS total_salary
FROM employees
WHERE salary > 3500
GROUP BY department;
```

Explicación

La consulta mostrará el salario total por departamento, pero incluirá sólo los salarios individuales por encima de $3,500 en la suma. Funciona de la siguiente manera.

Primero, por supuesto, selecciona los departamentos y utiliza `SUM()` con la columna de salario de la tabla `employees`. Eso ya lo has aprendido.

A continuación, utilice la cláusula `WHERE` para especificar los valores que desea incluir en la suma. En este caso, cuando la columna salario sea superior a 3.500. En otras palabras, la consulta sumará sólo los valores superiores a 3.500.

Por último, agrupe por departamento.

Salida

department	total_salary
Private Individuals	11,217.95

department	total_salary
Corporate	19,025.31

Estos totales incluyen ahora sólo los salarios superiores a 3.500 $. Compárelo con el resultado del undécimo ejemplo (que se muestra a continuación; tenga en cuenta que la ordenación es diferente) y verá que los totales son inferiores. Es lógico, ya que la salida de abajo también incluye salarios iguales o inferiores a $3.500.

department	total_salaries
Corporate	21,919.82
Private Individuals	17,072.92

17. Uso de COUNT(), WHERE y GROUP BY

Esta también es una de las consultas que le aconsejamos que incluya en su caja de herramientas SQL. Es similar a la anterior, ya que utiliza una función agregada. Este tipo de consulta se puede utilizar cuando se desea mostrar el número de ocurrencias de cada grupo.

Consulta

Code

```
SELECT
  department,
  COUNT(*) AS number_of_employees
FROM employees
WHERE salary > 3500
GROUP BY department;
```

Explicación

Esta consulta es similar a la anterior, sólo que utiliza la función agregada `COUNT()`. Su objetivo es mostrar el nombre del departamento y el número de empleados en ese departamento, pero cuenta sólo los empleados con un salario superior a $3,500.

Para conseguirlo, primero selecciona el departamento. Luego utilice `COUNT(*)` para contar todas las filas dentro de cada departamento. Cada fila equivale a un empleado. Podemos utilizar esta versión de la función `COUNT()` porque sabemos que no hay filas `NULL`.

Ahora, utilice `WHERE` para incluir en el conteo sólo a los empleados con salarios superiores a $3500.

Al final, sólo necesita agrupar los datos por departamento.

Salida

department	number_of_employees
Private Individuals	3
Corporate	4

La salida muestra que hay tres empleados en el departamento Privado Individualque cobran más de 3.500 $ y que hay cuatro empleados de este tipo en el departamento Corporativo.

Obviamente faltan algunos empleados, como debe ser. En uno de los ejemplos anteriores aprendimos que hay cinco empleados en cada departamento.

18. Acceso a Datos en Dos Tablas Utilizando INNER JOIN

Este tipo de consulta se utiliza cuando se desea acceder a datos de dos o más tablas. Le mostraremos INNER JOIN, pero no es el único tipo de join que puede utilizar.

He aquí un breve resumen de los tipos de join en SQL. Estos son los nombres completos de las uniones. Lo que se muestra entre paréntesis puede omitirse en la consulta y la unión funcionará sin él.

SQL Join Type	Description
(INNER) JOIN	Returns the matching values from both tables.
LEFT (OUTER) JOIN	Returns all the values from the left table and only the matching values from the right table.
RIGHT (OUTER) JOIN	Returns all the values from the right table and only the matching values from the left table.
FULL (OUTER) JOIN	Returns all the rows from both tables.
CROSS JOIN	Returns all combinations of all rows from the first and second table, i.e. the Cartesian product.

Consulta

Code

```
SELECT
  e.id,
  e.first_name,
  e.last_name,
  qs.q1_2022 + qs.q2_2022 + qs.q3_2022 + qs.q4_2022 AS total_sales_2022
FROM employees e
JOIN quarterly_sales qs
ON e.id = qs.employee_id;
```

Explicación

Esta consulta quiere mostrar el ID y el nombre de cada empleado, junto con sus ventas totales en 2022.

Para ello, utiliza `JOIN`, ya que los datos necesarios se encuentran en ambas tablas de nuestro conjunto de datos.

Empecemos a explicar la consulta con la cláusula `FROM`. Esto nos resulta familiar: para utilizar los datos de la tabla `employees` es necesario hacer referencia a ella en `FROM`. También damos a esta tabla un alias ('e'), para no tener que escribir el nombre completo de la tabla más adelante.

A continuación, utilizamos la palabra clave `JOIN` para unirnos a la segunda tabla. Para ello, hacemos referencia a la tabla `quarterly_sales` en `JOIN` y dándole el alias 'qs'.

Ahora viene la condición `ON`. Se utiliza para especificar las columnas en las que se unirán las dos tablas. Normalmente, son las columnas que almacenan los mismos datos en ambas tablas. En otras palabras, unimos las tablas sobre las claves primaria y foránea. Una clave primaria es una columna (o columnas) que define de forma exclusiva cada fila de la tabla. Una clave externa es una columna de la segunda tabla que hace referencia a la primera. En nuestro ejemplo, la columna `id` de la tabla `employees` es su clave principal. La columna `employee_id` de la tabla `quarterly_sales` es la clave externa, ya que contiene el valor de la columna `id` de la primera tabla.

Así que utilizaremos estas columnas en `ON`, pero también tenemos que especificar de qué tabla procede cada columna. Recuerda que hemos dado alias a nuestras tablas. Esto será útil aquí, ya que no necesitaremos escribir los nombres completos de las tablas - sólo una letra para cada tabla. Escribimos el alias de la primera tabla (en lugar de su nombre completo), los separamos con un punto, y luego el nombre de la columna. Ponemos el signo igual, el alias de la segunda tabla y el nombre de la columna.

Ahora que tenemos dos tablas unidas, podemos seleccionar cualquier columna de ambas tablas. Seleccionamos `id`, `first_name`, y `last_name` de `employees`. A continuación, añadimos cada columna de la tabla **ventas_trimestrales** que muestra las ventas trimestrales y la nombramos `total_sales_2022`. Cada columna de `SELECT` también tiene el alias de la tabla delante, con el alias y el nombre de la columna separados por un punto.

Nota: Al unir tablas, es aconsejable utilizar los nombres de tabla delante de los nombres de columna en `SELECT` es aconsejable. Así será más fácil determinar qué columna procede de qué tabla. Además, las tablas pueden tener columnas con el mismo nombre. Sin embargo, los nombres de tabla pueden resultar farragosos, por lo que también es aconsejable darles alias en `JOIN`. De esta forma, puede utilizar alias mucho más cortos (en lugar de los nombres completos de las tablas) delante de los nombres de las columnas.

Salida

id	first_name	last_name	total_sales_2022
8	Stephanie	Marx	22,993.23
4	Lucy	Patterson	30,595.57
10	Victoria	Pollock	6,770.16
1	Paul	Garrix	45,443.38
3	Matthias	Johnson	58,233.67
2	Astrid	Fox	20,312.48
7	Walter	Deer	62,774.59
6	Claudia	Conte	17,734.94
5	Tom	Page	28,637.65
9	Luca	Pavarotti	25,023.21

La salida lista a cada empleado y muestra sus ventas totales en 2022.

19. 19. Acceso a Datos en Dos Tablas Utilizando INNER JOIN y Filtrando Utilizando WHERE

Por supuesto, puede filtrar datos en tablas unidas de la misma forma que puede hacerlo con una sola tabla. Nuevamente necesitará la cláusula `WHERE` .

Consulta

Code

```
SELECT
  e.id,
  e.first_name,
  e.last_name,
  qs.q4_2022-qs.q3_2022 AS sales_change
FROM employees e
JOIN quarterly_sales qs
ON e.id = qs.employee_id
WHERE qs.q4_2022-qs.q3_2022 < 0;
```

Explicación

Hemos modificado la consulta anterior para mostrar la disminución de las ventas entre el tercer y el cuarto trimestre.

Así es como lo hicimos. Igual que hicimos antes, seleccionamos el ID y el nombre del empleado.

Restamos un trimestre de otro para calcular el cambio entre los trimestres. En este caso, es la columna con las ventas del cuarto trimestre menos las ventas del tercer

trimestre. Esta nueva columna se llama `sales_change` .

Las tablas se unen exactamente igual que en el ejemplo anterior.

Para mostrar sólo la disminución de las ventas, utilizamos la cláusula `WHERE` . En ella, volvemos a restar el tercer trimestre del cuarto y establecemos la condición de que el resultado tiene que ser inferior a cero, es decir, una disminución. Como has podido comprobar, `WHERE` aparece después de unir las tablas.

Salida

id	first_name	last_name	sales_change
8	Stephanie	Marx	-2,224.11
4	Lucy	Patterson	-5,151.55
1	Paul	Garrix	-21,233.46
3	Matthias	Johnson	-3,771.83
7	Walter	Deer	-189.32
6	Claudia	Conte	-1,500.44
9	Luca	Pavarotti	-1,138.55

La salida muestra todos los empleados que han tenido una disminución de ventas en el último trimestre y el importe de dicha disminución.

20. 20. Accediendo a Datos en Dos Tablas Usando INNER JOIN, Filtrando Usando WHERE, y Ordenando con ORDER BY

Probablemente notó que las salidas en nuestros dos últimos ejemplos están ordenadas un poco al azar. Esto no es algo que tenga que soportar - puede ordenar los datos con `ORDER BY` incluso cuando utilice dos tablas.

Consulta

Code

```
SELECT
  e.id,
  e.first_name,
  e.last_name,
  qs.q4_2022
FROM employees e
JOIN quarterly_sales qs
ON e.id = qs.employee_id
WHERE qs.q4_2022 > 5000
ORDER BY qs.q4_2022 DESC;
```

Explicación

La consulta no difiere mucho de la anterior. Volvemos a seleccionar el ID y el nombre del empleado. También añadimos las ventas del último trimestre del año. Las tablas se unen de la misma forma que antes. Utilizamos la cláusula `WHERE` para mostrar sólo las ventas trimestrales superiores a 5.000 dólares.

Además, queremos ordenar la salida. Esto no difiere de lo que aprendimos anteriormente: simplemente escriba el nombre de la columna en `ORDER BY` y ordénela como desee. En nuestro ejemplo, estamos ordenando de mayor a menor las ventas trimestrales.

Salida

id	first_name	last_name	q4_2022
7	Walter	Deer	14,445.12
3	Matthias	Johnson	8,451.51
5	Tom	Page	7,788.01
9	Luca	Pavarotti	7,648.90
2	Astrid	Fox	5,322.05

La salida muestra los cinco empleados cuyas ventas fueron superiores a $5,000 en los últimos tres meses de 2022.

De Consultas SQL Básicas a Maestro SQL

Si desea dominar SQL, debe sentirse cómodo utilizando estas 20 consultas SQL básicas. Estos son los fundamentos que le permitirán construir un sólido conocimiento de SQL.

> LearnSQL.es es una plataforma que te permite recorrer todos los temas de SQL y elegir el camino correcto para ti con la garantía de poder cambiar de opinión en cualquier momento sin consecuencias.

Este tipo de conocimiento se consigue con mucha práctica y experiencia. En otras palabras, simplemente necesitas escribir las consultas por tu cuenta. De esa manera, consolidarás todos los conceptos que aprendiste aquí. Por el camino, probablemente cometerás muchos errores. Esto es deseable, ya que no hay mejor forma de aprender que intentar corregir tus propios errores.

Para ello necesitarás muchos ejemplos de consultas. No, no es necesario que hagas tus propios ejemplos, como hicimos aquí. Puedes hacerlo si quieres. Pero ya lo hicimos por ti en nuestro SQL para principiantes curso.

Está repleto de ejemplos de consultas SQL básicas. Pruébalo, ¡y estamos seguros de que no te arrepentirás!

Tags: SQL Consultas De SQL

También te pueden interesar

una CTE
QL Server

so para escribir
SQL Server.



Cómo utilizar GROUP BY y ORDER BY en la misma consulta: Una Guía Detallada

GROUP BY y ORDER BY en una misma consulta puede ser complicado de usar. Desmenuzamos todos los matices y le mostramos ejemplos de cómo utilizar estas cláusulas SQL



Cómo utilizar COUNT() con GROUP BY: 5 Ejemplos Prácticos

Saber utilizar COUNT GROUP BY es esencial en SQL. Esta combinación permite a los usuarios extraer información y generar resúmenes estadísticos a partir de grandes conjuntos de datos.

Cómo Escribi Recursiva en

Una guía paso a p
CTEs recursivas e

Más información

Copyright ©2016-2026 **Vertabelo SA** Todos los derechos reservados